

Code Duplication Detection — Teknik Rapor

Proje: CodeDuplicationDetection

Tarih: Nisan 2026

Veri Seti: POJ-104 (C/C++ kaynak kodu, 104 problem sınıfı, ~52.000 dosya)

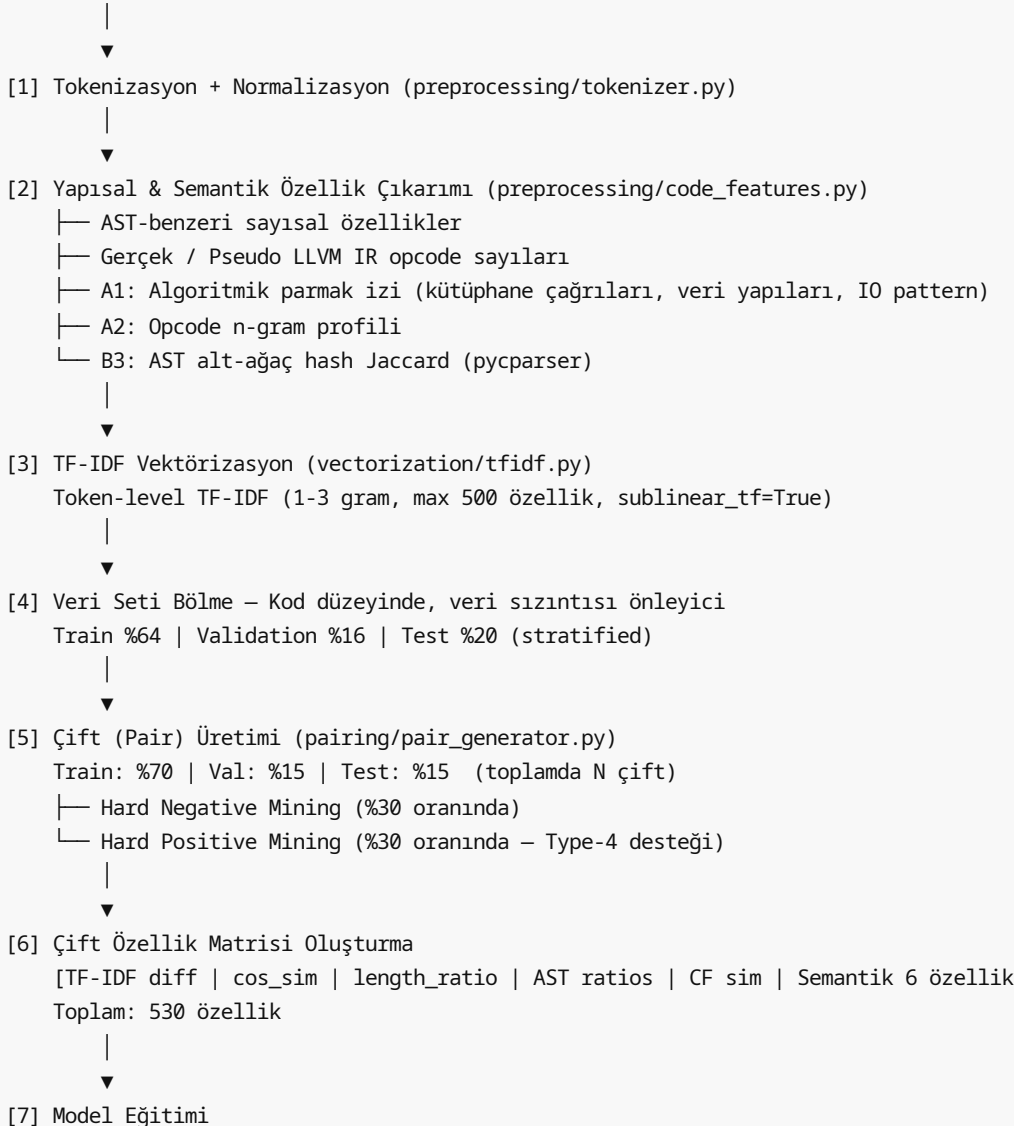
En İyi Model: XGBoost (exp_050) — Accuracy: %95.91 | F1: %95.87 | AUC-ROC: %99.36

1. Proje Genel Akışı

Sistem, iki C/C++ kod parçasını karşılaştırarak aralarında anlamsal (semantik) benzerlik bulunup bulunmadığını tahmin eden bir ikili sınıflandırma (binary classification) sistemidir. Tahmin edilen sınıflar şunlardır:

- 1 (Duplicate / Clone):** İki kod aynı problemi çözmekte; kaynak kodu, mantık veya algoritma düzeyinde benzeşmektedir.
- 0 (Not Duplicate):** İki kod farklı problemlere aittir.

Ham Kaynak Kod (C/C++)



```
XGBoost (varsayılan) | RandomForest | LinearSVM | MLP | Ensemble
├─ Semantik özelliklere 1000x ağırlık (feature_weights)
├─ Early Stopping (validation seti üzerinde, 30 round sabır)
└─
    ▼
[8] Değerlendirme & Deney Kayıt (utils/experiment_logger.py)
    Accuracy, F1, AUC-ROC, Confusion Matrix, Classification Report
    │
    ▼
[9] Çıktı
    experiments/exp_NNN_Model_Xk/
    ├── model.pkl, tfidf.pkl
    ├── metrics_train.json, metrics_test.json
    ├── classification_report_{train|test}.{txt|json}
    ├── confusion_matrix_{train|test}.png
    └── config.json, notes.txt
```

2. Veri Seti: POJ-104

Özellik	Değer
Kaynak	Peking Online Judge (POJ) arşivi
Problem Sayısı (Sınıf Sayısı)	104
Toplam Kod Dosyası	~52.000 (her problem için ~500 çözüm)
Dil	C / C++
Format	.txt dosyaları, data/poj104/<class_id>/<file>.txt
Görev	Aynı probleme ait iki kodun tespiti (klon tespiti)

Her bir `class_id` klasörü, aynı algoritmik problemi çözen farklı programcılarının kodlarını içermektedir. Aynı sınıftan iki kod → Pozitif çift (clone), farklı sınıflardan iki kod → Negatif çift (non-clone).

Veri Bölme Stratejisi

Veri sızıntısını (data leakage) önlemek için bölme **çift düzeyinde değil, kod düzeyinde** yapılır. Bir kodun eğitim setinde olması durumunda o kodun tüm çiftleri test setinde yer alamaz.

```
Tüm kodlar (52.000)
├─
├─ Train+Val (80%, stratified) → Train (64%) + Validation (16%)
└─ Test (20%)
```

Çift sayıları (1.000.000 çift için):

- **Eğitim çiftleri:** 700.000 (%70)
- **Validasyon çiftleri:** 150.000 (%15)
- **Test çiftleri:** 150.000 (%15)

3. Ön İşleme: Tokenizasyon ve Normalizasyon

preprocessing/tokenizer.py

Ham C/C++ kodu şu adımlardan geçirilir:

1. **Regex tokenizasyonu:** Tanımlayıcılar, sayılar, çok karakterli operatörler (&& , || , << , >> , ++ , ->), karşılaştırma operatörleri ve noktalama işaretleri regex ile ayrıştırılır.

2. **Normalizasyon:**

Token Türü	Dönüşüm
C/C++ anahtar sözcükleri (int, for, return, ...)	Olduğu gibi bırakılır
Sayısal değerler (42, 3.14, ...)	→ NUM
Döngü değişkenleri (i, j, k, n, m, ...)	Olduğu gibi bırakılır
Tek harfli identifier	→ VAR
Diğer tüm identifierlar	→ VAR
Operatörler ve noktalama	Olduğu gibi bırakılır

Bu normalizasyon, değişken isimlerinden bağımsız bir token temsili oluşturur; Type-1 ve Type-2 klonların TF-IDF katmanında yakalanmasını kolaylaştırır.

4. Özellik Mühendisliği (Feature Engineering)

Proje, iki kod arasındaki benzerliği 5 farklı özellik grubuna dayandırarak hesaplar. Toplam **530 özellik** çift başına üretilmektedir.

4.1 TF-IDF Token Farkı (500 özellik)

$|TF-IDF(code_A) - TF-IDF(code_B)| \rightarrow$ Seyrek (sparse) fark vektörü

- 1-3 gram token TF-IDF, max 500 özellik, sublinear_tf=True, min_df=3, max_df=0.95
- TF-IDF yalnızca **eğitim** verisi üzerinde fit edilir; validasyon/test sadece transform
- Yakalar:** Type-1 (birebir kopya), Type-2 (yeniden adlandırılmış değişkenler), Type-3 (küçük farklar içeren kopyalar)

4.2 Skaler / Çift Düzey Özellikler (30 özellik)

#	Özellik	Açıklama
1	cosine_similarity_token	İki TF-IDF vektörü arasındaki kosinüs benzerliği
2	length_ratio	$\min(\text{len}_A, \text{len}_B) / \max(\text{len}_A, \text{len}_B)$ — kod uzunluk oranı
3-23	AST feature ratios (21 özellik)	Her AST/IR sayısal özelliği için min/max oranı
24	cf_pattern_similarity	Kontrol akışı dizisinin bigram Jaccard benzerliği

25	semantic_library_call_jaccard	Kullanılan kütüphane fonksiyonlarının Jaccard benzerliği
26	semantic_data_struct_jaccard	Kullanılan veri yapılarının Jaccard benzerliği
27	semantic_io_pattern_jaccard	Giriş/Çıkış sırasının bigram Jaccard benzerliği
28	semantic_math_op_jaccard	Kullanılan matematiksel operatörlerin Jaccard benzerliği
29	semantic_opcode_ngram_cosine	Pseudo-IR opcode 2-gram profilinin kosinüs benzerliği
30	semantic_subtree_hash_jaccard	AST alt-ağaç hash kümesinin Jaccard benzerliği

4.3 AST-Benzeri Sayısal Özellikler (8 özellik)

preprocessing/code_features.py içinde regex ile çıkarılır:

Özellik	Hesaplama
branch_count	if, else, switch, case sayısı
loop_call_combined	for+while+do döngüsü + fonksiyon çağırısı sayısı
nesting_depth	Maksimum iç içe {} derinliği
operator_count	+, -, *, /, %, aritmetik operatör sayısı
return_count	return ifadesi sayısı
accumulator_pattern	+=, -=, x = x + y biçimi tespit (0/1)
param_count	İlk fonksiyon tanımındaki parametre sayısı
math_op_set_size	Kullanılan benzersiz matematiksel operatör tipi sayısı

4.4 LLVM IR Opcode Özellikleri (10 özellik)

Sistem önce clang++ ile gerçek LLVM IR derlemeyi dener. Başarısız olursa Pseudo-IR'ye geçer.

Opcode	Açıklama
load	Değişken okuma (gerçek SSA load)
store	Atama / değişken yazma
cmp	Karşılaştırma (icmp, fcmp)
math	Aritmetik (add, sub, mul, sdiv, srem, ...)
bitwise	Bit işlemleri (shl, and, or, xor, ...)
br	Dallanma (br, switch, ret, invoke)
alloc	Bellek tahsisi (alloca, malloc, new)
phi	SSA phi düğümü (yalnızca gerçek IR'de)
gep	getelementptr — dizi/işaretçi erişimi

call	Fonksiyon çağırısı
------	--------------------

4.5 A1: Algoritmik Parmak İzi (3 sayısal + frozen set)

Özellik	
library_call_count	Tanınan ~80 standart C/C++ kütüphane fonksiyonu kullanım sayısı
data_struct_count	Algılanan veri yapısı tipi sayısı (array, stack, queue, vector, map, linked_list, matrix, string)
io_pattern_length	Giriş/Çıkış sırası uzunluğu ("IIIOOO" → 6)
library_calls (set)	Kullanılan kütüphane fonksiyon kümesi
data_structs (set)	Kullanılan veri yapısı tipi kümesi
io_pattern (str)	"I" ve "O" karakterlerinden oluşan IO sırası
math_ops (set)	Kullanılan matematiksel operatör kümesi ({+, -, *})

4.6 A2: Opcode N-gram Profili

Pseudo-IR opcode sırası (store , cmp , math , br , call , ...) çıkarılır ve **2-gram frekansları** Counter objesi olarak saklanır. İki kod arasında kosinüs benzerliği hesaplanır.

Örnek: ['store', 'cmp', 'math', 'br'] → {'store-cmp': 1, 'cmp-math': 1, 'math-br': 1}

4.7 B3: AST Alt-ağaç Hash Jaccard

pycparser kütüphanesi ile C/C++ kodu gerçek AST'ye dönüştürülür. Weisfeiler-Lehman inspired hashing ile her alt-ağacın yapısal hash'i (id/değer bağımsız, yalnızca tip bazlı) çıkarılır. İki kodun hash kümeleri arasında Jaccard benzerliği hesaplanır.

5. Klon Tiplerine Göre Hangi Özellikler Yakalar?

Klon Tipi	Tanım	Yakalayan Özellikler
Type-1	Birebir kopya (yorum/boşluk farkı)	TF-IDF diff, cosine sim, length ratio
Type-2	Yeniden adlandırılmış değişkenler	TF-IDF (VAR normalizasyonu sayesinde), AST feature ratios
Type-3	Bazı satırlar eklenmiş/silinmiş (near-miss)	TF-IDF fark + CF pattern sim + AST ratios
Type-4	Aynı algoritmayı farklı uygulayan kodlar (iteratif vs özyinelemeli)	IR opcode oranları, A1 (kütüphane+veri yapısı+IO), A2 (opcode n-gram cosine), B3 (AST subtree Jaccard), accumulator pattern, math op set

Type-4 Ağırlıklandırma: XGBoost'ta son 30 özelliğe (AST + CF + semantik) `feature_weight=1000.0` uygulanarak model bu özelliklere 1000 kat daha fazla dikkat eder.

6. Çift Üretimi ve Hard Mining

pairing/pair_generator.py

Temel Çift Üretimi

- ~%50 pozitif (aynı sınıf), ~%50 negatif (farklı sınıf) çift üretilir.

Hard Negative Mining (%30 oranında)

- Negatif çiftlerin %30'u için kaynak kodla **benzer uzunluktaki** farklı sınıf kodu seçilir.
- Amacı: Modelin kolayca ayırt edemeyeceği aldatıcı negatif çiftler eklemek, karar sınırını keskinleştirmek.

Hard Positive Mining (%30 oranında — Type-4 desteği)

- Pozitif çiftlerin %30'u için aynı sınıf içinden **en farklı uzunluktaki** kod seçilir.
- Amacı: İterasyonel vs özimizelemeli gibi yapısal olarak farklı ama anlamsal olarak eşdeğer kodları çiftle; modeli Type-4 tanımaya zorla.

7. Modeller

7.1 XGBoost (Varsayılan & En İyi Model)

models/xgboost.py — GPUGBClassifier (XGBClassifier wrapper)

Parametre	Değer	Açıklama
n_estimators	3000 (max, early stopping ile durur)	Ağaç sayısı tavanı
max_depth	6	Daha sığ ağaçlar → daha az ezber
learning_rate	0.03	Yavaş öğrenme → daha iyi genelleme
subsample	0.7	Her ağaç için veri örnekleme oranı
colsample_bytree	0.6	Her ağaç için özellik örnekleme oranı
colsample_bylevel	0.6	Seviye başına özellik randomizasyonu
min_child_weight	8	Minimum yaprak örnek sayısı
gamma	0.3	Katı dal budama eşiği
reg_alpha	0.1	L1 — seyreklik cezası
reg_lambda	1.0	L2 — ağırlık küçültme
early_stopping_rounds	30	Validation logloss iyileşmezse dur
eval_metric	logloss	Erken durma metriği
tree_method	hist	Histogram tabanlı, hızlı
device	cuda / cpu (otomatik)	GPU desteği
feature_weights	Son 30 özellik × 1000.0	Semantik özellik önceliklendirme

7.2 Random Forest

`models/random_forest.py` — `RandomForestClassifier(n_estimators=500, max_depth=20, class_weight='balanced')`

7.3 Linear SVM

`models/linear_svm.py` — `CalibratedClassifierCV(LinearSVC(...))` — Olasılık tahmini için kalibrasyonlu.

7.4 Deep Learning (MLP)

`models/dl_model.py` — `SimpleMLP` — PyTorch, 2 gizli katman (512→128), Dropout(%30), BCEWithLogitsLoss.

Giriş (530 boyut) → Linear(512) → ReLU → Dropout(0.3) → Linear(128) → ReLU → Dropout(0.3) → Linear(1) → Sigmoid

Intel XPU (Arc GPU) için IPEX + BF16 mixed precision desteği mevcuttur.

7.5 Ensemble (Soft Voting)

`models/ensemble.py` — XGBoost + Random Forest + Calibrated LinearSVM'in soft voting (olasılık ortalaması) birleşimi.

8. Hyperparameter Tuning

`utils/hyperparameter_tuner.py` — **Optuna** ile bayesian optimizasyon (TPE Sampler).

- `--tune` flag'i ile etkinleştirilir
- Default: 30 deneme (trial), `--tune-trials N` ile değiştirilebilir
- Hedef metrik: **F1 skoru** (3-fold CV üzerinde)
- Desteklenen modeller: XGBoost, Random Forest, Linear SVM

9. Dizin Yapısı ve Sınıf Referansı

```
CodeDuplicationDetection/
├─ main.py                      # Ana eğitim orkestratörü
├─ requirements.txt             # Bağımlılıklar
├─ data/
│   └─ poj104/                  # Eğitim veri seti (104 sınıf, ~52.000 dosya)
│       └─ 1/                   # Problem 1'e ait kodlar
│           └─ 1001.txt
│               └─ ...
│                   └─ ...
├─ preprocessing/
│   └─ tokenizer.py             # tokenize(), normalize_tokens()
│       └─ code_features.py     # extract_all_features(), _extract_single()
│                               # FEATURE_NAMES listesi (21 özellik adı)
```

```

|                                     # cf_pattern_similarity()
|
|— vectorization/
|   |— tfidf.py                     # build_tfidf_vectorizer()
|                                     # build_char_tfidf_vectorizer()
|
|— pairing/
|   |— pair_generator.py            # generate_pairs() – çift matrisi üretimi
|                                     # _jaccard_sim(), _counter_cosine_sim()
|
|— models/
|   |— xgboost.py                  # GPUXGBClassifier, build_xgboost()
|   |— random_forest.py            # build_random_forest()
|   |— linear_svm.py               # build_linear_svm()
|   |— dl_model.py                 # SimpleMLP, PyTorchClassifier, build_dl_model()
|   |— ensemble.py                 # build_voting_ensemble()
|
|— utils/
|   |— experiment_logger.py         # generate_experiment_name(), save_experiment()
|   |                                     # save_cv_results()
|   |— feature_pipeline.py         # build_pair_vector() – demo & test için
|   |— hyperparameter_tuner.py     # tune_hyperparameters() (Optuna)
|   |— llvm_compiler.py            # extract_true_llvm_ir() (clang++ wrapper)
|   |— test_automation.py          # run_automation() – otomatik test
|
|— experiments/                    # Deney sonuçları (50 deney)
|   |— exp_NNN_Model_Xk/
|       |— model.pkl               # Eğitilmiş model
|       |— tfidf.pkl               # TF-IDF vectorizer
|       |— config.json             # Deney parametreleri
|       |— metrics_train.json      # Eğitim metrikleri
|       |— metrics_test.json       # Test metrikleri
|       |— classification_report_{train|test}.{txt|json}
|       |— confusion_matrix_{train|test}.png
|       |— notes.txt               # Zamanlama özeti
|
|— test_clones/                    # Otomasyon test verisi (20 problem)
|   |— prob_01/ ... prob_20/
|
|— test_results/                   # Otomasyon çıktıları
|   |— run_<timestamp>/
|       |— report.json
|       |— summary.txt
|
|— web_demo/
|   |— app.py                      # FastAPI sunucusu + SHAP açıklamaları
|   |— index.html                  # Web arayüzü

```

10. Nasıl Çalıştırılır? — Tüm Parametreler

Gereksinimler

```
pip install -r requirements.txt
# LLVM IR desteği için:
sudo apt install clang # veya brew install llvm
```

10.1 Model Eğitimi (main.py)

```
python main.py [PARAMETRELER]
```

Parametre	Tip	Varsayılan	Açıklama
--model	str	xgboost	Model seçimi: xgboost, random_forest, linear_svm, ensemble, dl_model
--dataset	str	data/poj104	Veri seti dizini
--pairs	int	800000	Üretilecek toplam çift sayısı
--test-size	float	0.2	Test bölümü oranı
--seed	int	42	Rastlantı tohumu (reproducibility)
--tune	flag	kapalı	Optuna ile hiperparametre arama aktif
--tune-trials	int	30	Optuna deneme sayısı
--device	str	auto	cpu, cuda, xpu, auto
--cv	flag	kapalı	Stratified K-Fold CV modu
--cv-folds	int	5	CV kat sayısı
--cv-pairs	int	None	CV başına çift sayısı (None → --pairs değeri)

Örnek komutlar:

```
# Hızlı deneme (200k çift, XGBoost)
python main.py --pairs 200000

# En iyi sonuç (1M çift, GPU)
python main.py --pairs 1000000 --device cuda

# RandomForest ile Optuna tuning
python main.py --model random_forest --tune --tune-trials 50

# 5-Fold cross-validation
python main.py --cv --cv-folds 5 --pairs 400000

# Deep Learning, Intel XPU
python main.py --model dl_model --device xpu
```

10.2 Otomatik Test (utils/test_automation.py)

```
python -c "from utils.test_automation import run_automation; run_automation()"
# veya
python utils/test_automation.py
```

- En son deneyin modeli otomatik olarak yüklenir.
- test_clones/ dizinindeki tüm intra-problem çiftleri (pozitif) ve bazı inter-problem çiftleri (negatif) test edilir.
- Sonuçlar test_results/run_<timestamp>/ altına kaydedilir.

Parametre	Varsayılan	Açıklama
test_dir	test_clones	Test verisi dizini
threshold	0.90	Pozitif tahmin eşiği (> 0.90 → Duplicate)

10.3 Web Demo (web_demo/app.py)

```
uvicorn web_demo.app:app --reload --port 8000
# veya belirli bir deney için:
EXP_ID=50 uvicorn web_demo.app:app --reload --port 8000
```

- GET / — Web arayüzü (index.html)
- POST /predict — JSON: {"code1": "...", "code2": "..."} → {"probability": 0.97, "prediction": "Duplicated", "shap": {...}}

EXP_ID ortam değişkeni ayarlanmazsa en son deney otomatik yüklenir.

11. Çıktıların Anlamı

experiments/exp_NNN_Model_Xk/

Dosya	İçerik	Nasıl Okunur
config.json	Model adı, çift sayısı, model & TF-IDF parametreleri, toplam özellik sayısı	Deneyin tam konfigürasyonu
metrics_train.json	accuracy, f1_score (train seti)	Modelin eğitim verisine ne kadar uyduğu
metrics_test.json	accuracy, f1_score, auc_roc (test seti)	Modelin genelleme performansı — ana başarı metriği
classification_report_test.txt	0/1 sınıfları için precision, recall, F1, support	Eşit dengesiz dağılımda sınıf bazlı performans
classification_report_test.json	Aynı rapor, makine okuma formatı	Program entegrasyonu için

confusion_matrix_test.png	TP/FP/TN/FN görsel	Hata tiplerinin görsel analizi
confusion_matrix_train.png	Eğitim confusion matrix	Ezberleme tespiti (train >> test ise sorun var)
model.pkl	Pickle ile kaydedilmiş model	<code>pickle.load()</code> ile yükleme
tfidf.pkl	TF-IDF vektörizer durumu	Model ile birlikte her zaman yüklenmeli
notes.txt	Deney tarihi + faz bazlı süre (dakika)	Aşama bazında süre analizi

metrics_test.json değerlerinin yorumu

Metrik	Formül	İdeal	exp_050
accuracy	$(TP+TN) / \text{Toplam}$	>0.95	0.9591
f1_score	$2 \times (P \times R) / (P+R)$	>0.95	0.9587
auc_roc	TPR vs FPR eğrisi altı alan	>0.98	0.9936

- **accuracy ≈ train_accuracy:** Aşırı öğrenme (overfitting) yoktur.
- **auc_roc > 0.99:** Model ayırt edici gücü çok yüksek (olasılık sıralaması neredeyse mükemmel).

Web Demo /predict cevabı

```
{
  "probability": 0.9712,
  "prediction": "Duplicated",
  "shap": {
    "base_value": 0.4821,
    "features": [
      {"feature": "semantic_library_call_jaccard", "value": 1.0, "shap_value": 0.312},
      {"feature": "cosine_similarity_token", "value": 0.87, "shap_value": 0.215},
      ...
    ]
  }
}
```

- probability : 0-1 arası klon olasılığı. > 0.90 → Duplicate.
- prediction : "Duplicated" veya "Not Duplicated"
- shap.base_value : Modelin genel ortalama tahmini (base rate)
- shap.features : Hangi özelliğin kararı hangi yönde ne kadar etkilediği (SHAP açıklanabilirliği)
 - shap_value > 0 : Bu özellik tahmini "Duplicate" yönüne itti
 - shap_value < 0 : Bu özellik tahmini "Not Duplicate" yönüne itti
 - value : O çift için özelliğin ham değeri

test_results/run_<timestamp>/summary.txt

```
Precision: 0.9523 → Duplicate dediğimizin %95.23'ü gerçekten Duplicate
Recall: 0.9712 → Gerçek Duplicate'lerin %97.12'si tespit edildi
```

F1-Score: 0.9616 → Precision ve Recall'ın harmonik ortalaması
TP: 450, FP: 23, TN: 201, FN: 13

12. Deneysel Süreç Özeti (50 Deney)

Deney Grubu	Model	Çift Sayısı	Bulgu
exp_001-004	Random Forest	20k-1M	Büyük veriyle ciddi iyileşme; 400k ile platoya ulaşıyor
exp_005-006	Random Forest	400k	Özellik ekleme testleri
exp_007-012	Linear SVM	400k-2M	RF'den genelde daha iyi; büyük veriyle ciddi kazanım
exp_013-028	XGBoost	10k-800k	XGBoost en yüksek AUC-ROC'u verdi
exp_029-032	Deep Learning / Ensemble	1k	MLP düşük veriye duyarlı; Ensemble rekabetçi
exp_033-044	XGBoost	400k-1M	Semantik özellikler (A1/A2/B3) eklendi
exp_045	XGBoost CV5	1M	5-fold cross-validation
exp_046-050	XGBoost	1M	Regularizasyon fine-tuning; en iyi: exp_050

En iyi deney: exp_050_XGBoost_1000k

- Train Accuracy: **99.06%** | Test Accuracy: **95.91%** | AUC-ROC: **99.36%**

13. Genel Özet

CodeDuplicationDetection projesi, C/C++ kod çiftlerini **anlamsal düzeyde** karşılaştırarak klon tespiti yapan, ikili sınıflandırma (binary classification) temelli bir makine öğrenmesi sistemidir.

Projenin temel katkıları:

- Çok katmanlı özellik mühendisliği:** Token TF-IDF'den gerçek LLVM IR opcode'larına kadar 530 boyutlu zengin bir temsil.
- Type-4 klon desteği:** Algoritmik parmak izi (A1), opcode n-gram profili (A2) ve AST alt-ağaç hashing (B3) ile birbiriyle yapısal olarak farklı fakat semantik olarak eşdeğer kodları tespit etme.
- Hard Mining stratejisi:** Hem negatif hem pozitif örneklerde güçlü seçimi yoluyla karar sınırının keskinleştirilmesi.
- Veri sızıntısız bölme:** Kod düzeyinde ayırma, gerçek dünyaya yakın değerlendirme garantisi.
- Kapsamlı deney takibi:** 50 deney, otomatik isimlendirme, JSON/PNG çıktı.
- Üretim altyapısı:** FastAPI + SHAP açıklanabilirliği ile web demo; Optuna ile hiperparametre optimizasyonu.

14. İyileştirme ve Geliştirme Önerileri

14.1 Özellik Mühendisliği Geliştirmeleri

A) Program Dependence Graph (PDG) / Control Flow Graph (CFG) Özellikleri

Şu an kontrol akışı `string` olarak temsil ediliyor. `networkx` veya `pycparser` 'dan türetilecek gerçek bir CFG/PDG ile daha güçlü bir grafik kökeli özellik eklenebilir. İki CFG arasındaki `edit distance` veya `graph kernel similarity` Type-4 tespiti için güçlü bir sinyal olacaktır.

B) Code2Vec / Code Embeddings

Microsoft'un `CodeBERT` veya `UniXcoder` gibi büyük dil modelleri (LLM), kodu anlamsal vektörlere dönüştürebilir. Her kodun embedding vektörü çıkarılıp çiftler arasında cosine/L2 mesafesi hesaplanabilir. Bu özellik mevcut pipeline'a kolayca eklenebilir (yeni bir sütun olarak).

C) Graph Neural Networks (GNN) ile AST Encoding

AST node'ları grafik düğümlere dönüştürülerek TreeLSTM veya GNN mimarileriyle öğrenilen temsil, sabit boyutlu AST özelliklerinden çok daha zengin bilgi taşır. Özellikle Type-4 klonlar için büyük fark yaratabilir.

D) Halstead Karmaşıklık Metrikleri

Operatör/operand sayılarından türetilen Halstead hacim, güçlük ve çaba metrikleri, aynı algoritmik problemi çözen kodların benzer karmaşıklık tablolarına sahip olacağı varsayımıyla iyi bir Type-4 sinyali olabilir.

14.2 Özellik Seçimi (Feature Selection)

Şu an 530 özelliğin tamamı modele verilmektedir. Özellik seçimi hem generalleşmeyi artırabilir hem de tahmin süresini kısaltabilir:

- **SHAP tabanlı seçim:** Web demo zaten SHAP değerlerini hesaplıyor. Eğitim setinde SHAP importance skorlarına göre en az katkı sağlayan özellikler (örneğin alt %20) kesilebilir.
- **Recursive Feature Elimination (RFE):** `sklearn.feature_selection.RFECV` ile cross-validation içinde en önemli özellik alt kümesi bulunabilir.
- **Mutual Information:** `sklearn.feature_selection.mutual_info_classif` ile her özelliğin hedef değişkene katkısı ölçülebilir.
- **Boyut azaltma (PCA/UMAP):** TF-IDF 500 boyutunu 50-100 boyuta indirgeyerek bilgi kaybı olmadan hesaplama yükü düşürülebilir. Özellikle sparse TF-IDF farkı kompres edilebilir.

14.3 Model Mimarisi Geliştirmeleri

A) Siamese Network Mimarisi

Mevcut yapı iki kodu birleştirerek tek vektör oluşturur. Gerçek bir Siamese ağı iki kodu bağımsız encode ederek sonra kontrastif kayıp fonksiyonu (contrastive loss veya triplet loss) ile öğrenir. Daha güçlü genelleme ve öğrenilmiş semantic embedding sağlar.

B) Gradient Boosting Alternatifleri

LightGBM veya CatBoost, XGBoost ile benzer performans gösterirken çok daha hızlı eğitim sunabilir. Özellikle 1M+ çift senaryosunda LightGBM'in leaf-wise büyüme stratejisi avantajlı olabilir.

C) Stacking Ensemble

Mevcut soft voting yerine XGBoost + RF + SVM'in çıktılarını bir meta-learner (logistic regression veya başka bir XGBoost) ile birleştiren **stacking** mimarisi, voting'den genellikle daha yüksek performans verir.

D) Transformer Tabanlı Vektörleştirme

BERT tokenizer ve attention mekanizması ile token sırasını dikkate alan bir embedding oluşturmak, n-gram TF-IDF'e kıyasla sıra bağımlı (order-aware) özellikler sağlar. CodeBERT ön-eğitimli ağırlıkları ile fine-tuning yapılabilir.

14.4 Eğitim Süreci Geliştirmeleri

A) Daha Büyük Veri Seti

POJ-104'e ek olarak BigCloneBench veya Google Code Jam veri setleri eklenebilir. Cross-dataset genelleme test edilmelidir.

B) Curriculum Learning

Eğitim sürecinde önce kolay (Type-1/2) sonra zorlu (Type-4) örnekler sunulursa model kademeli olarak daha iyi öğrenebilir.

C) Self-Supervised Ön-Eğitim

Kod veri setinin etiketli kısmını kullanmadan önce kodu maskeleye/tamamlama görevi ile ön-eğitim yapılabilir (CodeBERT tarzı). Bu representation daha zengin semantik bilgi taşır.

D) Cross-Language Genelleme

Java, Python gibi farklı dillerde de klon tespiti yapabilmek için dil-agnostik özellikler (opcode, CFG, veri yapısı) ön plana çıkarılıp dil bazlı özellikler azaltılabilir.

14.5 Altyapı & Üretim İyileştirmeleri

- **Model versiyonlama:** MLflow veya DVC ile deney takibi zenginleştirilebilir.
- **REST API cache:** Aynı kod çiftleri için tekrar tahmin yapmamak üzere Redis cache eklenebilir.
- **Asenkron batch prediction:** FastAPI'ya büyük liste girişi için arka plan görevi (BackgroundTask) eklenebilir.
- **Confidence calibration:** XGBoost'un çıkardığı olasılıklar `CalibratedClassifierCV` ile kalibre edilerek daha güvenilir olasılık tahminleri elde edilebilir.

Bu rapor, CodeDuplicationDetection projesinin kaynak kodundan otomatik analiz edilerek Nisan 2026'da üretilmiştir.